

SENIOR SWE INTERVIEW SERIES

Module 4: HTTP (1.1 / 2 / 3, HTTPS, QUIC)

Interview-focused networking, as asked at Google, Meta, Amazon, Microsoft, Uber, Netflix, Cloudflare, Stripe, LinkedIn & Cisco

Part of: Computer Networking for Senior Software Engineer Interviews

Built July 02, 2026

MODULE 4 — HTTP (1.1 / 2 / 3, HTTPS, QUIC)

HTTP evolution is the single best-scoring interview narrative: every version exists to kill a specific bottleneck of the previous one. Tell it as a story of head-of-line blocking moving down the stack until QUIC finally killed it.

Topic 4.1 — HTTP/1.1

1. Why Interviewers Ask This

It's still everywhere (most internal service traffic, ALB → target defaults, curl). Interviewers test whether you understand its concurrency model — because its limits explain *why* HTTP/2 and connection pools exist.

2. Core Concept

Text-based request/response over TCP. One request **at a time** per connection (send request → wait full response → next). Concurrency = open more connections (browsers: 6 per host). Persistent connections (keep-alive) default since 1.1.

3. Internal Working

- Message framing: headers terminated by `\r\n\r\n`; body length via `Content-Length` or `Transfer-Encoding: chunked` (length-prefixed chunks, enables streaming without knowing size upfront).
- Pipelining (sending multiple requests before responses) exists in the spec but is dead in practice: responses must return in order → HoL blocking + broken middleboxes. Say this — it's a known trap.
- Headers are uncompressed plain text, resent fully on every request (cookies! often 1–2KB per request).

4. Packet Flow Explanation

```
[TCP handshake 1 RTT] [TLS 1-2 RTT]
GET /a ----->
<----- 200 /a      (connection idle-waits)
GET /b ----->      only now can /b start
<----- 200 /b
Serial: total = N * (RTT + server_time). Browsers parallelize with
6 connections => 6x handshakes, 6x slow starts, 6x TLS state.
```

5. ASCII Diagram

```
HTTP/1.1, one connection:  [req A][----resp A----][req B][--resp B--]
                           ^ B waits: app-level HoL blocking
Browser workaround:  conn1 [A.....]  conn2 [B.....]  ... x6
Server side effect:  10k users x 6 conns = 60k sockets
```

6. Real Production Example

Sharding domains (`img1.cdn.com` ... `img4.cdn.com`) was a standard 2010s hack to exceed the 6-connection cap — now an *anti-pattern* under HTTP/2 (splits one warm connection into many cold ones). Interviewers love asking why the best practice inverted.

7. Advantages

- Dead simple: debuggable with telnet/netcat; every proxy/LB/tool understands it.
- No stream-level state machine; failure isolation per connection.

8. Trade-offs

- Application-layer HoL blocking → latency serialization.
- Header overhead (uncompressed, repeated cookies) — significant for API-heavy pages.
- Connection storms: concurrency scales via sockets, multiplying handshakes, slow starts, and server memory.

9. Common Mistakes

- Claiming HTTP/1.1 can't reuse connections (keep-alive is default).
- Confusing pipelining (dead) with multiplexing (HTTP/2).
- Forgetting chunked encoding — “how do you stream a response of unknown length?” is a common probe.

10. Performance Impact

Page with 50 assets, RTT 50ms, 6 connections: $\geq \text{ceil}(50/6)$ serial rounds $\approx 9 \times (50\text{ms} + \text{server}) \approx 500\text{ms} +$ just in round trips. Same page on HTTP/2 $\approx 1\text{--}2$ rounds. This arithmetic is a favorite interview exercise.

11. Common Interview Questions

1. How does the client know where a response ends? (Content-Length / chunked / connection close.)
2. Why exactly 6 connections per host, and what does that cost?
3. Why did HTTP/1.1 pipelining fail?

12. Follow-up Questions

- “What's request smuggling?” → CL/TE ambiguity between front proxy and backend parsing the same bytes differently — the darkest corner of 1.1 framing (also Module 9-adjacent).
- “When is 1.1 still the right choice?” → simple internal RPC, LB-to-backend hops with low concurrency, maximum tool compatibility.

13. Debugging Scenarios

- Sporadic hangs at high concurrency → pool exhaustion (all 6/host busy); look for connection-wait metrics in client pools.
- Truncated bodies → mismatched Content-Length vs actual bytes (broken proxy or app bug).

14. Best Practices

- Tune client pool size to real concurrency; enable keep-alive both hops (LB → backend too, not just client → LB).
- Set explicit read/idle timeouts; 1.1 has no ping frame to detect dead peers.

15. Practice Questions

1. API client makes 200 sequential calls/page-load over one keep-alive connection, RTT 30ms. Minimum added latency vs HTTP/2 multiplexing? ($\sim 200 \times 30\text{ms} = 6\text{s}$ vs $\sim 30\text{ms} + \text{server time}$.)
2. Explain why `Connection: close` responses can't be truncated undetectably but keep-alive ones need Content-Length.

Topic 4.2 — HTTP/2

1. Why Interviewers Ask This

It's the default on every CDN/LB and the transport under gRPC. Interviewers check: binary framing, multiplexing, HPACK, flow control — and crucially, *the TCP HoL-blocking flaw that QUIC fixes*.

2. Core Concept

Same HTTP semantics, new wire format: a **binary framing layer** multiplexes many concurrent **streams** (request/response pairs) over **one TCP connection**. Adds header compression (HPACK), stream priorities, per-stream flow control, server push (deprecated in practice).

3. Internal Working

- Everything is a frame: `HEADERS`, `DATA`, `SETTINGS`, `WINDOW_UPDATE`, `RST_STREAM`, `PING`, `GOAWAY`. Each frame carries a stream ID; frames of different streams interleave freely.
- **HPACK**: static table (61 common headers) + dynamic table (per-connection, learned) + Huffman coding → repeated headers cost $\sim 1\text{--}2$ bytes. Stateful: both ends must stay in sync (why HTTP/3 needed QPACK).
- Flow control at two levels (connection + stream) — receivers grant window via `WINDOW_UPDATE`; a slow-consumed stream can't drown the connection.
- `RST_STREAM` cancels one request without killing the connection (impossible in 1.1 — you had to drop the socket).

4. Packet Flow Explanation

```
one TCP+TLS connection:
-> HEADERS(s1: GET /a) HEADERS(s3: GET /b) HEADERS(s5: GET /c)  (no waiting)
<- DATA(s3) DATA(s1) DATA(s3) DATA(s5) DATA(s1)...         (interleaved)
All three requests in flight simultaneously; responses arrive as ready.
BUT: all frames ride ONE TCP byte stream =>
one lost TCP segment stalls ALL streams until retransmitted (TCP HoL).
```

5. ASCII Diagram

```

HTTP/2 streams:  s1 ====A====      multiplexed
                  s3 ==B==        --> [ frame frame frame ] -> one TCP stream
                  s5 =====C=====
TCP loses 1 segment:
  TCP buffer: [ok][ok][LOST][ok][ok]  kernel delivers nothing past the hole
=> s1, s3, s5 ALL freeze although their bytes arrived. (QUIC's raison d'être)

```

6. Real Production Example

gRPC is HTTP/2: streams = RPCs, trailers carry status, PING frames = keepalive, GOAWAY = graceful drain. When Kubernetes services behind L4 LBs pile all gRPC onto one connection to one pod, load skews — the famous “gRPC doesn’t balance through NLB” issue (fix: L7 LB or client-side balancing) — a top interview story.

7. Advantages

- One warm connection: one handshake, one TLS session, one slow-start ramp, big shared cwnd.
- No app-level HoL blocking; request cancellation; ~90% header compression.

8. Trade-offs

- TCP-level HoL blocking under loss (measurably worse than 1.1×6 at >~2% loss).
- Stateful compression + multiplexing = more complex proxies (HPACK bomb, stream-abuse DoS like Rapid Reset — CVE-2023-44487, worth citing).
- One connection = one congestion-control context: a single lossy path event hits everything.

9. Common Mistakes

- “HTTP/2 solves head-of-line blocking” — it solves *application*-level HoL, and *moves* the problem to TCP. This nuance is THE most common interview differentiator.
- Thinking HTTP/2 changes methods/status codes (semantics unchanged).
- Advocating server push (deprecated; Chrome removed it — replaced by 103 Early Hints).

10. Performance Impact

Typical page loads: 10–30% faster than 1.1 on clean networks. Under 2%+ loss, can be *slower* than 1.1×6 (six independent streams = six chances to not be blocked). Headers: 85–90% compression → matters hugely for cookie-heavy APIs and mobile.

11. Common Interview Questions

1. How does multiplexing actually work at the frame level?
2. Explain HPACK and why compression is stateful.
3. Where does HoL blocking remain, exactly?

12. Follow-up Questions

- “Why does gRPC need PING frames if TCP has keep-alive?” → verifies the HTTP/2 layer/peer process is alive and keeps L7 proxy idle timers fresh (TCP KA is invisible to L7).

- “What is GOAWAY for?” → graceful shutdown: server announces last-processed stream ID so clients retry newer requests elsewhere — key to zero-downtime deploys.
- “Rapid Reset attack?” → open+RST streams at line rate; server does work per stream, attacker pays ~nothing; mitigations: pending-stream limits, rate-limit RSTs.

13. Debugging Scenarios

- All requests slow simultaneously in spikes → single-connection loss events; check TCP retransmits on that flow; consider connection count >1 or HTTP/3.
- gRPC calls hang while TCP looks healthy → stream-level flow-control window exhaustion (slow reader); inspect WINDOW_UPDATE flow (h2 debug logs/ `GODEBUG=http2debug=2`).

14. Best Practices

- Stop 1.1-era hacks: no domain sharding, no asset concatenation/spriteing (hurts caching).
- Cap concurrent streams sensibly (`SETTINGS_MAX_CONCURRENT_STREAMS` ~100–250); protect proxies against RST floods.
- For gRPC at scale: L7 load balancing or client-side LB with subsetting.

15. Practice Questions

1. 100 API calls, RTT 40ms, server 5ms each: total time on 1.1×6 vs h2×1? (1.1: $\text{ceil}(100/6) \times 45\text{ms} \approx 765\text{ms}$; h2: ~45ms + serialization — order of magnitude.)
2. Why can one slow-reading client stall its own downloads but not other clients on your h2 server? (Per-stream + per-connection flow control isolates; different clients = different connections.)

Topic 4.3 — HTTP/3 & QUIC

1. Why Interviewers Ask This

The current frontier — asked constantly at Google (invented it), Cloudflare, Meta (>75% of their mobile traffic is QUIC). Tests whether you keep current and truly understood the HoL story.

2. Core Concept

QUIC = a reliable, multiplexed, always-encrypted transport built in **userspace over UDP**, with TLS 1.3 fused into its handshake. **HTTP/3** = HTTP semantics mapped onto QUIC streams. Key wins: 1-RTT (or 0-RTT) combined transport+crypto handshake, **independent stream delivery** (no transport HoL), **connection migration** via connection IDs.

3. Internal Working

- Streams are first-class in the transport: each stream has its own ordering/reassembly. Loss in stream A never blocks stream B (the fix HTTP/2 couldn't have on TCP).
- Packet numbers are **never reused** (retransmitted data goes in a new packet) → unambiguous RTT/loss accounting.
- Everything after the first flight is encrypted **including headers/ACKs** → middleboxes can't ossify the protocol (explicitly designed goal; the “spin bit” is the one deliberate concession to network operators).

- Connection ID ≠ 4-tuple → client hops WiFi → LTE and the connection *continues* (mobile!).
- QPACK replaces HPACK: header compression redesigned so table updates can't deadlock independent streams.
- Congestion control: pluggable, userspace (CUBIC/BBR), iterated weekly instead of per-kernel-release.

4. Packet Flow Explanation

```
Cold connect, HTTP/3:
C->S  QUIC Initial (contains TLS ClientHello)          \
S->C  Initial+Handshake (ServerHello, cert, ...)      } 1 RTT total
C->S  Handshake finish + HTTP request                  /
Repeat visit (0-RTT): C->S  Initial + 0-RTT DATA (request!) -> 0 RTT
vs TCP+TLS1.3: 1 (TCP) + 1 (TLS) = 2 RTT ; TLS1.2: 3 RTT
Loss: pkt carrying stream-A bytes lost -> stream B,C keep delivering;
      only A waits for its retransmission.
```

5. ASCII Diagram

```

HTTP/2 stack          HTTP/3 stack
[ HTTP/2 streams ]    [ HTTP/3 ]
[ TLS 1.3           ]  [ QUIC: streams+crypto+CC+reliab. ]
[ TCP (ordered!) ] <-HoL [ UDP ]
[ IP ]               [ IP ]
one hole in TCP = all stop    one hole = only that stream waits

Migration: conn-id C7 over (wifi ip) -> (lte ip) same C7 => survives
```

6. Real Production Example

- Google: ~all Search/YouTube mobile traffic; reported single-digit-% latency wins and big tail-latency wins on lossy networks.
- Meta: Instagram/Facebook apps on QUIC — up to 20% fewer video stalls reported.
- Cloudflare/Fastly/Akamai: HTTP/3 default. iOS/Android HTTP stacks negotiate it automatically via **Alt-Svc** /HTTPS DNS records.

7. Advantages

- 1-RTT/0-RTT setup; zero transport HoL; migration across networks; unblockable evolution (encrypted transport headers); per-stream everything.

8. Trade-offs

- **CPU**: ~2× TCP+TLS cost per byte today (userspace, fewer NIC offloads; improving via UDP GSO, crypto offload).
- UDP hostility: ~3–8% of networks block/throttle it → mandatory TCP fallback, negotiated via Alt-Svc (first visit usually starts on h2!).
- 0-RTT data is **replayable** → only idempotent requests allowed in 0-RTT (interview trap).
- Harder ops: encrypted transport = no passive middlebox diagnostics; per-packet decryption needed in tooling (qlog exists for this).

9. Common Mistakes

- “QUIC is unreliable because UDP” — QUIC is fully reliable; UDP is just the substrate beneath its own reliability layer.
- Believing first-ever connection starts on h3 (it’s discovered via Alt-Svc/HTTPS-record; first contact often h2).
- Ignoring the 0-RTT replay caveat.

10. Performance Impact

Biggest wins where it hurts most: lossy/mobile/long-RTT users (tail latency, video rebuffering). Clean datacenter LAN: negligible or negative (CPU). This asymmetry — “who benefits” — is exactly what interviewers want articulated.

11. Common Interview Questions

1. What problem does HTTP/3 solve that HTTP/2 fundamentally couldn’t? (Transport HoL — because it required changing TCP itself.)
2. Why over UDP instead of a new transport protocol? (Middleboxes/NATs drop unknown IP protocols; kernels ossify — userspace over UDP deploys everywhere today.)
3. How does connection migration work and why does mobile care?

12. Follow-up Questions

- “How does a server share load across cores if there’s no accept()?” → connection-ID-aware routing/ eBPF steering; LBs route by connection ID, not 4-tuple.
- “What is the amplification limit?” → $\leq 3\times$ bytes until client address validated (anti-DDoS).
- “Why QPACK vs HPACK?” → HPACK’s synchronized dynamic table would reintroduce cross-stream blocking; QPACK splits encoder/decoder streams with controlled dependencies.

13. Debugging Scenarios

- Users behind corporate firewalls slower than home users → h3 blocked → TCP fallback; measure fallback rate; ensure fallback path is warm.
- Load imbalance across QUIC servers behind L4 LB → LB hashing 4-tuple breaks migration; need QUIC-aware (conn-ID) routing.
- Debugging tools: `curl --http3`, Wireshark with TLS keys, qlog/qvis traces.

14. Best Practices

- Enable h3 at the edge (CDN does the hard part); keep origin on h2/h1 (edge translates).
- Restrict 0-RTT to safe/idempotent endpoints.
- Monitor protocol mix + fallback rates as first-class SLO inputs.

15. Practice Questions

1. Mobile user on train, IP changes every few minutes, downloading a large file. Compare experience on h2 vs h3. (h2: connection dies per IP change — reconnect + TLS + slow start + range request; h3: seamless migration.)

2. Why might your p50 not improve after enabling h3 while p95 improves 30%? (Clean-path users were fine; loss/RTT-affected tail gains from HoL removal + faster handshakes.)

Topic 4.4 — HTTPS (TLS over HTTP)

1. Why Interviewers Ask This

Every system design has TLS termination somewhere, and its placement (edge vs LB vs pod) is a real architectural decision. Handshake internals are covered in Module 9; here: HTTP-specific implications.

2. Core Concept

HTTPS = HTTP over TLS. Provides confidentiality, integrity, and server authentication (client auth = mTLS, standard for service-to-service). Port 443. Practically: SNI routes by hostname before decryption; ALPN negotiates h1/h2/h3 during the handshake.

3. Internal Working

- **SNI** (Server Name Indication): client states target hostname in ClientHello (plaintext, unless ECH) → lets one IP serve thousands of certs (CDNs) and lets L4 proxies route without decrypting.
- **ALPN**: protocol negotiation inside TLS — this is how h2 is chosen (there's no **Upgrade** dance for HTTPS).
- Termination points: edge/CDN (fastest handshakes near users), LB (common), or end-to-end to the pod (zero-trust/mTLS, e.g., service meshes with sidecars).

4. Packet Flow Explanation

```
TCP SYN/SYNACK/ACK (1 RTT)
ClientHello [SNI: api.example.com, ALPN: h2,http/1.1]
ServerHello + cert + Finished (TLS 1.3: 1 RTT; resumption/0-RTT less)
-> encrypted HTTP/2 begins (SETTINGS exchange, then requests)
Total cold cost: 2 RTT (TCP+TLS1.3) before first request byte.
```

5. ASCII Diagram

```
user --TLS--> [CDN edge: terminate] --TLS2--> [LB] --mTLS--> [service]
      cert: example.com      internal CA      SPIFFE identity
"TLS everywhere" = 3 separate TLS sessions, 3 cert stories.
SNI: [ClientHello: api.example.com] -> L4 router picks backend WITHOUT keys
```

6. Real Production Example

Cloudflare serves millions of domains from one IP set — SNI selects the cert. Post-2016 “encrypt everything”: Let’s Encrypt made certs free/automated; internal traffic followed via service meshes (Istio/Linkerd mTLS) after the Snowden-era revelations that Google’s inter-DC links were tapped — Google then encrypted *all* internal traffic (great interview anecdote).

7. Advantages

- Table stakes security; HTTP/2+ effectively requires it (browsers only do h2/h3 over TLS); enables ALPN, HSTS, modern web features (browsers gate APIs on secure contexts).

8. Trade-offs

- Handshake RTT + CPU (~1–2% of modern server CPU — cite this; “TLS is expensive” is outdated), certificate lifecycle ops (expiry outages are a top-10 industry incident class!).
- Termination placement trade-off: terminate early = performance + WAF/caching possible; terminate late = stronger zero-trust posture. Middle answer: re-encrypt hop-by-hop.

9. Common Mistakes

- “TLS makes everything slow” — handshake latency matters (mitigate: resumption, edge termination, h3), steady-state CPU is small (AES-NI).
- Forgetting internal hops: TLS user → LB but plaintext LB → service is a common real-world gap interviewers probe.
- Not knowing what SNI/ALPN do (both are load-bearing for CDNs and h2 adoption).

10. Performance Impact

Cold handshake: +1 RTT (TLS 1.3) — from 150ms-away users that's +150ms on first request; session resumption + keep-alive amortize to ~zero. OCSP stapling avoids a client-side revocation fetch. Certificate chain > initial cwnd (~14KB) adds an RTT — real tuning item.

11. Common Interview Questions

1. Where do you terminate TLS in your design, and why?
2. What are SNI and ALPN? What breaks without them?
3. How do you do TLS for 10k customer custom domains on your SaaS? (Automated issuance via ACME, SNI serving, cert storage/rotation.)

12. Follow-up Questions

- “mTLS between services — what does it buy over network ACLs?” → identity-based auth (works across IP churn), encryption in transit, per-service authz.
- “What is ECH?” → encrypts SNI itself using a key from DNS — closes the last plaintext metadata leak.

13. Debugging Scenarios

- `openssl s_client -connect host:443 -servername host` — the debugging swiss knife (verify cert chain, ALPN result, expiry).
- Intermittent handshake failures for some clients only → missing intermediate cert (some clients cache it, some don't) or SNI-less legacy clients hitting default cert.

14. Best Practices

- Automate issuance/renewal (ACME), alert at 30/14/7 days; staple OCSP; TLS 1.3 + resumption; keep chains small.
- Terminate at edge for latency, re-encrypt internally (mTLS via mesh for service identity).

15. Practice Questions

1. p99 for first-time mobile users is 900ms; returning users 180ms. Break down where the 720ms goes and cut it. (DNS + TCP + TLS cold + slow start; fixes: edge termination, resumption tickets, h3 0-RTT, smaller cert chain, preconnect hints.)
2. Design cert strategy for a mesh of 500 microservices. (Private CA, short-lived certs [~24h], SPIFFE-style identities, sidecar-managed rotation — no manual certs ever.)

Topic 4.5 — Persistent Connections, Compression, Multiplexing (Cross-Version Synthesis)

1. Why Interviewers Ask This

These three mechanisms are *the* levers of HTTP performance, and interviewers ask them as comparisons across versions (“how did each version handle X?”).

2. Core Concept

- **Persistence:** 1.0 opt-in (`Connection: keep-alive`) → 1.1 default → h2/h3 one long-lived multiplexed connection per origin.
- **Compression:** bodies (gzip → brotli → zstd, via `Accept-Encoding` / `Content-Encoding`) and headers (none → HPACK → QPACK).
- **Multiplexing:** none (1.1) → streams over TCP (h2) → streams over QUIC (h3, loss-isolated).

3. Internal Working

Body compression is end-to-end, negotiated per request; brotli ~15–20% smaller than gzip for text (precompress static assets at max level; dynamic at speed-optimized levels — brotli 4–5, gzip 6). Never compress already-compressed media (JPEG/MP4) — wasted CPU, occasionally bigger. **BREACH caveat:** compressing attacker-influenced + secret data in the same body leaks secrets via size — why compressing responses with CSRF tokens needs care (senior flex).

4. Packet Flow Explanation

The full budget of a modern first page view:

```
DNS (0-100ms, cached=0) -> TCP 1 RTT -> TLS 1 RTT -> request
-> HTML (14KB fits initial cwnd = 1 RTT) -> parse -> 20 subresources
  h1: 6 conns x more handshakes + serial rounds
  h2: same conn, all 20 in flight now      h3: same minus HoL risk
Compression: 300KB JS -> 80KB brotli => 3-4 fewer RTTs of slow-start ramp
```

5. ASCII Diagram

	persistence	header compr.	multiplexing	HoL location
HTTP/1.0	per-request(!)	none	none	connection
HTTP/1.1	keep-alive	none	none(6 conns)	application
HTTP/2	1 conn/origin	HPACK	streams/TCP	transport(TCP)
HTTP/3	1 conn/origin	QPACK	streams/QUIC	none (per-stream)

6. Real Production Example

Cookie-heavy enterprise apps: 2KB of cookies × 100 API calls = 200KB of *upload* headers on 1.1 — on asymmetric mobile uplinks this dominates. HPACK collapses repeats to bytes. This “headers cost upload bandwidth” angle is an underused interview point that lands well.

7. Advantages

Persistence amortizes handshakes + keeps cwnd warm; compression trades cheap CPU for scarce RTTs; multiplexing removes serialization.

8. Trade-offs

Persistence = server memory per idle conn + idle-timeout races (Module 2); compression = CPU + BREACH-class risks; multiplexing = shared-fate on one connection + more complex flow control.

9. Common Mistakes

- Compressing per-request at max level for dynamic content (CPU burn); not precompressing static.
- Keep-alive on client but LB idle-timeout shorter → reset storms (the classic).
- Assuming multiplexing helps upload-bound or single-large-file workloads (it doesn't; it helps many-small-objects).

10. Performance Impact

Rule-of-thumb stack for a global API: edge termination (−1−2 RTT), h2/h3 (−N×RTT serialization), brotli (−60−80% bytes), resumption (−1 RTT), 0-RTT (−1 RTT). Cumulative: cold 6-RTT mobile fetch → ~2 RTT.

11. Common Interview Questions

1. Compare how each HTTP version achieves concurrency.
2. Gzip vs brotli vs zstd — when each? (br for static text, zstd rising for dynamic/speed, gzip = compatibility floor.)
3. Why do repeated headers matter and how did h2 fix them?

12. Follow-up Questions

- “Why not compress everything at max?” → CPU per request; latency added by compression itself; diminishing returns past mid levels.
- “Where does multiplexing hurt?” → lossy paths on h2 (shared TCP), and fairness/prioritization complexity.

13. Debugging Scenarios

- High TTFB only for first request per user burst → cold connections; check keep-alive/idle timeouts and connection reuse rates at each hop.
- CPU spike on LB after enabling brotli-11 for API responses → dynamic max-level compression; drop to 4–5 or offload to edge.

14. Best Practices

- Reuse connections at every hop (client → CDN → LB → service); align idle timeouts increasing toward the server.

- Precompress static (br-11 + gzip-9 fallbacks); dynamic at moderate levels; **Vary: Accept-Encoding**.
- One origin (no sharding) for h2/h3; use priorities/early hints for critical assets.

15. Practice Questions

1. Your mobile API p95 improved 40% after “one infra change” — candidates: enabling h2 at LB, enabling gzip, moving TLS to edge. Which is most plausible for a 100-small-calls workload and why?
2. Compute upload header cost: 1.8KB cookies × 150 requests on h1 vs h2 (~270KB vs ~2KB after first request).

MODULE 4 — One-Page Cheat Sheet

THE NARRATIVE	1.1: serial per conn (app HoL) -> 6 conns hack h2: frames+streams over 1 TCP (HoL moved to TCP) + HPACK h3/QUIC: streams over UDP, loss-isolated, 0/1-RTT, migration
COLD RTTs	TCP1 + TLS1.3=1 + req = 2 RTT h3 = 1 RTT 0-RTT/resume = 0-1
FRAMING	1.1: text, Content-Length chunked (smuggling risk: CL/TE) h2: HEADERS/DATA/SETTINGS/RST_STREAM/PING/GOAWAY frames
KEY H2 FACTS	HPACK stateful; per-stream+conn flow control; GOAWAY drains; Rapid Reset attack; gRPC = h2 + trailers + PING keepalive
KEY H3 FACTS	pkt numbers never reused; all-encrypted transport; conn-ID migration; QPACK; 3x anti-amplification; Alt-Svc discovery; 0-RTT = idempotent only; TCP fallback mandatory
HTTPS	SNI (route before decrypt) ALPN (chooses h2) terminate early + re-encrypt cert expiry = top outage class
COMPRESSION	brotli static-11 / dynamic 4-5; never media; BREACH caveat
NUMBERS	6 conns/host (h1) ~100 streams (h2) headers ~90% HPACK brotli ~-20% vs gzip TLS CPU ~1-2% h3 CPU ~2x TCP

MODULE 4 — Top Interview Questions

1. Tell the 1.1 → 2 → 3 story strictly in terms of where head-of-line blocking lives. (The single highest-yield answer in this module.)
2. What happens when one TCP packet is lost under HTTP/2 with 10 active streams?
3. Why is QUIC in userspace over UDP? Three distinct reasons. (Middlebox traversal, kernel ossification, deploy velocity.)
4. Whiteboard every RTT from typed URL to first byte: cold vs warm vs 0-RTT.
5. Where do you terminate TLS and why — walk edge/LB/pod trade-offs.
6. How does gRPC use HTTP/2 primitives? What breaks behind an L4 LB?
7. Why did domain sharding become an anti-pattern?
8. 0-RTT: what's the attack and the rule? (Replay → idempotent-only.)

MODULE 4 — Common Mistakes

- “h2 fixes HoL blocking” without the TCP caveat — the #1 filter.

- Confusing pipelining with multiplexing; thinking h2 changed semantics.
- Forgetting Alt-Svc (first contact isn't h3); ignoring UDP-blocked fallback.
- TLS-is-slow mythology; unmanaged cert expiry; plaintext internal hops.
- Compressing media / max-level dynamic compression; sharding under h2.

MODULE 4 — Mock Interview (15 min)

Q1. “Your dashboard makes 300 small API calls. Users on hotel WiFi (2% loss, 120ms RTT) report 15s loads; office users 1.5s. Fix it.” *Strong answer:* diagnose: h1 pool serialization $\times$ RTT + loss-triggered stalls; short-term: batch/coalesce calls, enable h2 (one warm conn, multiplexed), edge-terminate TLS; then h3 for the lossy tail (per-stream loss isolation); measure protocol mix + p95 by network type; mention that h2 alone may underperform at 2% loss $\rightarrow$ h3 is the real fix for that cohort.

Q2. “We enabled HTTP/2 to origin and latency got worse under load. Why might that be?” *Strong answer:* all traffic collapsed onto few TCP connections: (a) LB $\rightarrow$ origin single-conn bottleneck + one CC context, (b) loss on that path stalls everything (TCP HoL), (c) per-stream flow-control windows too small for BDP, (d) origin concurrency limits tuned per-connection. Fixes: multiple h2 conns, window tuning, or keep h1.1 pool LB $\rightarrow$ origin (common CDN default!) while h2 client-side.

Q3. “Sketch how a request travels when the user has h3 cached via Alt-Svc but the corporate firewall silently drops UDP.” *Strong answer:* client races/falls back (happy-eyeballs-style or timeout) $\rightarrow$ TCP+TLS+h2; note the failure cost if fallback is timeout-based (hundreds of ms), why browsers cache “h3 broken here,” and why you monitor fallback rate per ASN.